OpenLane - Automatic RTL to GDSII flow

- Input files should be in a directory `designs/<design name>`
- Design directory should be organised as:
  1. `src/*.v`: directory containing verilog files
  2. `src/*.sdc`: clock and other timing specification
  3. `config.json`: configuration file to specify user inputs
  4. `pin_order.cfg`: specify which pins of your top module go to which boundary of the floorplan
- An example `adder` is in the directory: `/cad/examples/designs`

Tutorial 10 report
EE5311 (Digital IC design)

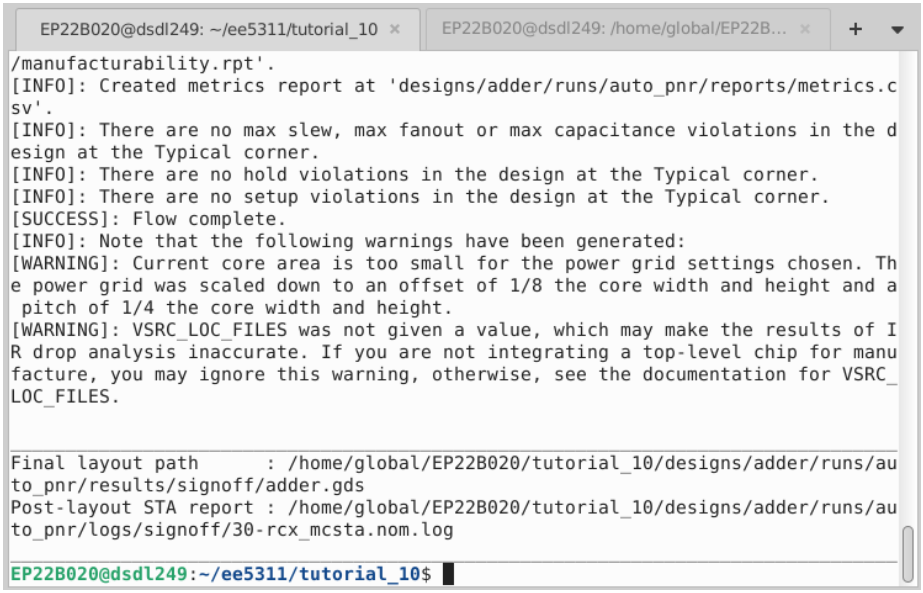
- Amogh G. Okade (EP22B020)

Generic Adder (used in slides)

- Verilog code used –

```
module adder (output reg[16:0] sum, input[15:0] a, input[15:0] b, input
cin, input clk);
    logic[15:0] a_q, b_q;
    logic cin_q;
    logic[16:0] sum_d;
    assign sum_d = {1'b0, a_q} + {1'b0, b_q} + {16'b0, cin_q};
    always @ (posedge clk) begin
        a_q <= a;
        b_q <= b;
        cin_q <= cin;
        sum <= sum_d;
    end
endmodule
```

- Part of output shown on running `openlane.sh design adder` –



A similar terminal log is required to ensure the successful completion of the openlane bash script for all the other adders.

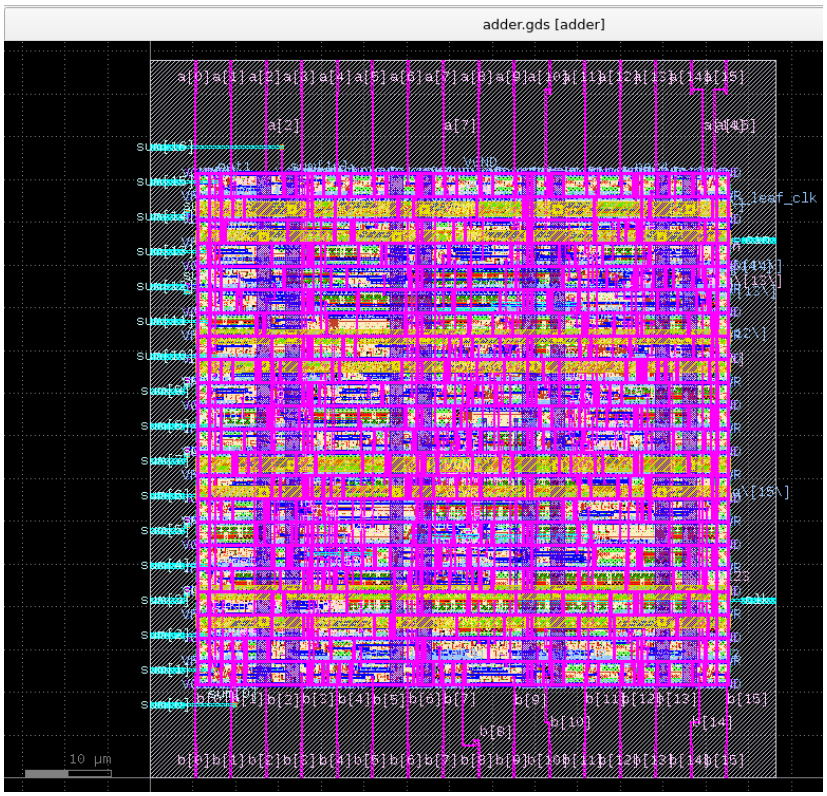
- Maximum clock frequency (approx.) = 0.217 GHz

```
set_units -time ns
create_clock [get_ports clk]  -name core_clock -period 4.6
```

```
=========================================================================================
report_worst_slack -max (Setup)
=========================================================================================
worst slack 0.07

=========================================================================================
report_worst_slack -min (Hold)
=========================================================================================
worst slack 0.33
```

- Auto-generated layout viewed in klayout –



16-bit ripple carry adder

- Verilog code used –

```
module rc16 (output reg[16:0] sum, input[15:0] a, input[15:0] b, input cin,
input clk);
    logic [15:0] a_q, b_q, co_d;
    logic cin_q;
    logic [16:0] sum_d;

    assign sum_d[0] = a_q[0] ^ b_q[0] ^ cin_q;
    assign co_d[0]  = (a_q[0] & b_q[0]) | (a_q[0] & cin_q) | (b_q[0] &
cin_q);
    genvar x;
    generate
        for (x = 1; x <= 15; x = x + 1) begin : ripple            //label
needed, otherwise errors
            assign sum_d[x] = a_q[x] ^ b_q[x] ^ co_d[x-1];
            assign co_d[x]  = (a_q[x] & b_q[x]) | (a_q[x] & co_d[x-1]) |
(b_q[x] & co_d[x-1]);
        end
    endgenerate
    assign sum_d[16] = co_d[15];

    always @ (posedge clk) begin
        a_q <= a;
```

```
        b_q <= b;
        cin_q <= cin;
        sum <= sum_d;
    end
endmodule
```

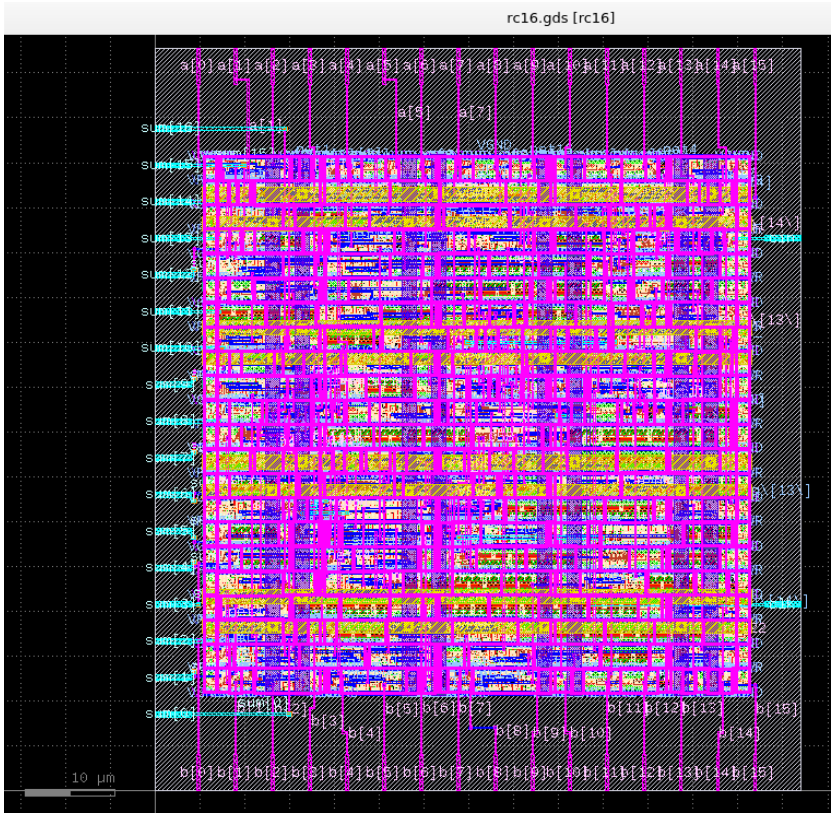
- Maximum clock frequency (approx.) = 0.169 GHz

```
set_units -time ns
create_clock [get_ports clk]  -name core_clock -period 5.9
```

```
=======================================
report_worst_slack -max (Setup)
=======================================
worst slack 0.02

=======================================
report_worst_slack -min (Hold)
=======================================
worst slack 0.37
```

- Auto-generated layout viewed in klayout –



16-bit Brent Kung adder

- Verilog code used –

```
module bk16 (output reg[16:0] sum, input[15:0] a, input[15:0] b, input cin,
input clk);
    logic[15:0] a_q, b_q, co_d, p, g;
    logic cin_q, p4, g4;
    logic[16:0] sum_d;
    logic[7:0] p1, g1;
    logic[3:0] p2, g2;
    logic[1:0] p3, g3;

    genvar x;
    generate
        for (x = 0; x <= 15; x = x + 1) begin : pggen
            assign p[x] = a_q[x] ^ b_q[x];
            assign g[x] = a_q[x] & b_q[x];
        end
```

```
    endgenerate

    genvar x1;
    generate
        for (x1 = 0; x1 <= 7; x1 = x1 + 1) begin : pg1
            assign p1[x1] = p[2*x1+1] & p[2*x1];
            assign g1[x1] = g[2*x1+1] | (p[2*x1+1]&g[2*x1]);
        end
    endgenerate

    genvar x2;
    generate
        for (x2 = 0; x2 <= 3; x2 = x2 + 1) begin : pg2
            assign p2[x2] = p1[2*x2+1] & p1[2*x2];
            assign g2[x2] = g1[2*x2+1] | (p1[2*x2+1]&g1[2*x2]);
        end
    endgenerate

    genvar x3;
    generate
        for (x3 = 0; x3 <= 1; x3 = x3 + 1) begin : pg3
            assign p3[x3] = p2[2*x3+1] & p2[2*x3];
            assign g3[x3] = g2[2*x3+1] | (p2[2*x3+1]&g2[2*x3]);
        end
    endgenerate

    assign p4 = p3[1] & p3[0];
    assign g4 = g3[1] | (p3[1]&g3[0]);

    assign co_d[0] = g[0] | (p[0] & cin_q);
    assign co_d[1] = g1[0] | (p1[0] & cin_q);
    assign co_d[2] = g[2] | (p[2] & co_d[1]);
    assign co_d[3] = g2[0] | (p2[0] & cin_q);
    assign co_d[4] = g[4] | (p[4] & co_d[3]);
    assign co_d[5] = g1[2] | (p1[2] & co_d[3]);
    assign co_d[6] = g[6] | (p[6] & co_d[5]);
    assign co_d[7] = g3[0] | (p3[0] & cin_q);
    assign co_d[8] = g[8] | (p[8] & co_d[7]);
    assign co_d[9] = g1[4] | (p1[4] & co_d[7]);
    assign co_d[10] = g[10] | (p[10] & co_d[9]);
    assign co_d[11] = g2[2] | (p2[2] & co_d[7]);
    assign co_d[12] = g[12] | (p[12] & co_d[11]);
    assign co_d[13] = g1[6] | (p1[6] & co_d[11]);
    assign co_d[14] = g[14] | (p[14] & co_d[13]);
    assign co_d[15] = g4 | (p4 & cin_q);

    assign sum_d[0] = p[0] ^ cin_q;
    genvar y;
    generate
        for (y = 1; y <= 15; y = y + 1) begin : suu
            assign sum_d[y] = p[y] ^ co_d[y-1];
        end
    endgenerate
    assign sum_d[16] = co_d[15];

    always @ (posedge clk) begin
        a_q <= a;
        b_q <= b;
        cin_q <= cin;
        sum <= sum_d;
    end
endmodule
```

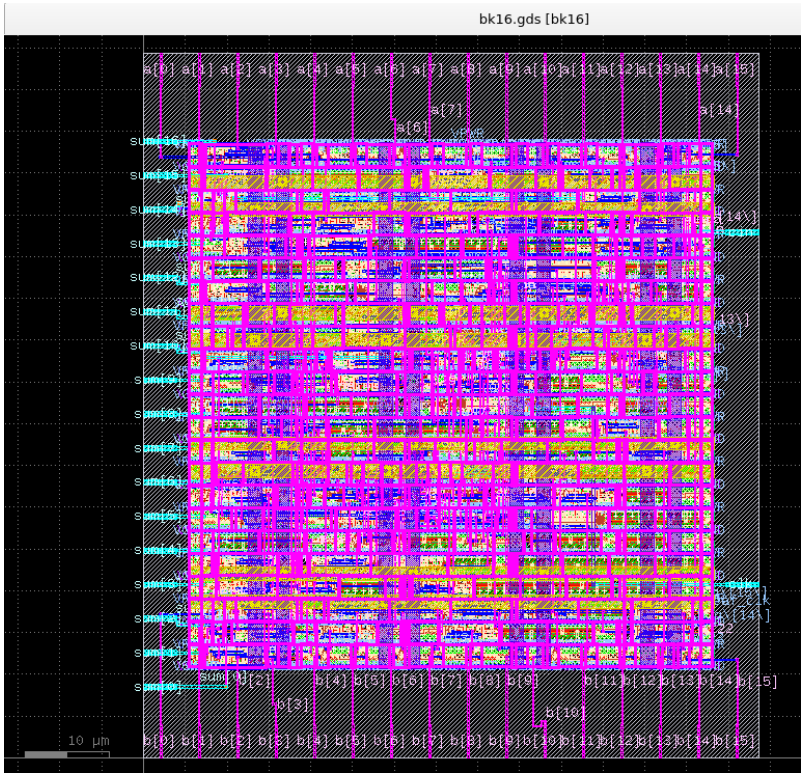
- Maximum clock frequency (approx.) = 0.196 GHz

```
set_units -time ns
create_clock [get_ports clk]  -name core_clock -period 5.1
```

```
=================================
report_worst_slack -max (Setup)
=================================
worst slack 0.11

=================================
report_worst_slack -min (Hold)
=================================
worst slack 0.34
```

- Auto-generated layout viewed in klayout –



16-bit Kogge Stone adder

- Verilog code used –

```
module ks16 (output reg[16:0] sum, input[15:0] a, input[15:0] b, input cin,
input clk);
    logic[15:0] a_q, b_q, co_d, p, g;
    logic cin_q;
    logic[16:0] sum_d;
    logic[14:0] p1, g1;
    logic[13:0] p2, g2;
    logic[11:0] p3, g3;
    logic[7:0] p4, g4;

    genvar x;
    generate
        for (x = 0; x <= 15; x = x + 1) begin : pggen
            assign p[x] = a_q[x] ^ b_q[x];
            assign g[x] = a_q[x] & b_q[x];
        end
    endgenerate

    genvar x1;
    generate
        for (x1 = 0; x1 <= 14; x1 = x1 + 1) begin : pg1
```

```
        assign p1[x1] = p[x1+1] & p[x1];
        assign g1[x1] = g[x1+1] | (p[x1+1]&g[x1]);
    end
endgenerate

assign p2[0] = p1[1] & p[0];
assign g2[0] = g1[1] | (p1[1]&g[0]);
genvar x2;
generate
    for (x2 = 1; x2 <= 13; x2 = x2 + 1) begin : pg2
        assign p2[x2] = p1[x2+1] & p1[x2-1];
        assign g2[x2] = g1[x2+1] | (p1[x2+1]&g1[x2-1]);
    end
endgenerate

assign p3[0] = p2[2] & p[0];
assign g3[0] = g2[2] | (p2[2]&g[0]);
assign p3[1] = p2[3] & p1[0];
assign g3[1] = g2[3] | (p2[3]&g1[0]);
genvar x3;
generate
    for (x3 = 2; x3 <= 11; x3 = x3 + 1) begin : pg3
        assign p3[x3] = p2[x3+2] & p2[x3-2];
        assign g3[x3] = g2[x3+2] | (p2[x3+2]&g2[x3-2]);
    end
endgenerate

assign p4[0] = p3[4] & p[0];
assign g4[0] = g3[4] | (p3[4]&g[0]);
assign p4[1] = p3[5] & p1[0];
assign g4[1] = g3[5] | (p3[5]&g1[0]);
assign p4[2] = p3[6] & p2[0];
assign g4[2] = g3[6] | (p3[6]&g2[0]);
assign p4[3] = p3[7] & p2[1];
assign g4[3] = g3[7] | (p3[7]&g2[1]);
genvar x4;
generate
    for (x4 = 4; x4 <= 7; x4 = x4 + 1) begin : pg4
        assign p4[x4] = p3[x4+4] & p3[x4-4];
        assign g4[x4] = g3[x4+4] | (p3[x4+4]&g3[x4-4]);
    end
endgenerate

assign co_d[0] = g[0] | (p[0] & cin_q);
assign co_d[1] = g1[0] | (p1[0] & cin_q);
assign co_d[2] = g2[0] | (p2[0] & cin_q);
assign co_d[3] = g2[1] | (p2[1] & cin_q);
assign co_d[4] = g3[0] | (p3[0] & cin_q);
assign co_d[5] = g3[1] | (p3[1] & cin_q);
assign co_d[6] = g3[2] | (p3[2] & cin_q);
assign co_d[7] = g3[3] | (p3[3] & cin_q);
genvar c;
generate
    for (c = 8; c <= 15; c = c + 1) begin : cgen
        assign co_d[c] = g4[c-8] | (p4[c-8] & cin_q);
    end
endgenerate

assign sum_d[0] = p[0] ^ cin_q;
genvar y;
generate
    for (y = 1; y <= 15; y = y + 1) begin : suu
        assign sum_d[y] = p[y] ^ co_d[y-1];
    end
endgenerate
assign sum_d[16] = co_d[15];
```

```
    always @ (posedge clk) begin
        a_q <= a;
        b_q <= b;
        cin_q <= cin;
        sum <= sum_d;
    end
endmodule
```

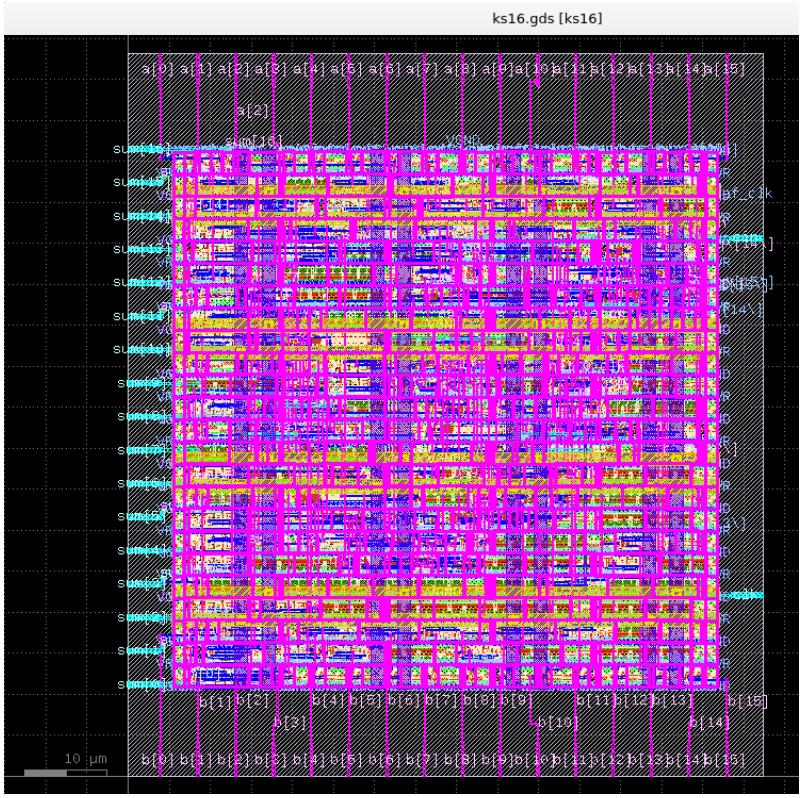
- Maximum clock frequency (approx.) = 0.212 GHz

```
set_units -time ns
create_clock [get_ports clk]  -name core_clock -period 4.7
```

```
==========================
report_worst_slack -max (Setup)
==========================
worst slack 0.02

==========================
report_worst_slack -min (Hold)
==========================
worst slack 0.35
```

- Auto-generated layout viewed in klayout –



Conclusion

- We can say that the speed of the adders varies as – **generic adder > Kogge Stone > Brent Kung > Ripple carry adder**.
  This goes to show that the generic adder is implemented in hardware to be as fast as it can.
- This also gives intuition on the fact that Verilog actually translates into hardware, since more number of lines/operations in code doesn't necessarily translate to more time taken by the adder, and that the actual availability of the signals (time of availability) is indeed important to the Verilog implementation.